

Scan2BIM Automation Platform: HLD

1. Project Overview

The **Scan2BIM automation platform** transforms raw telecom site point-cloud scan data into **Building Information Models / BIM outputs** with minimal manual intervention.

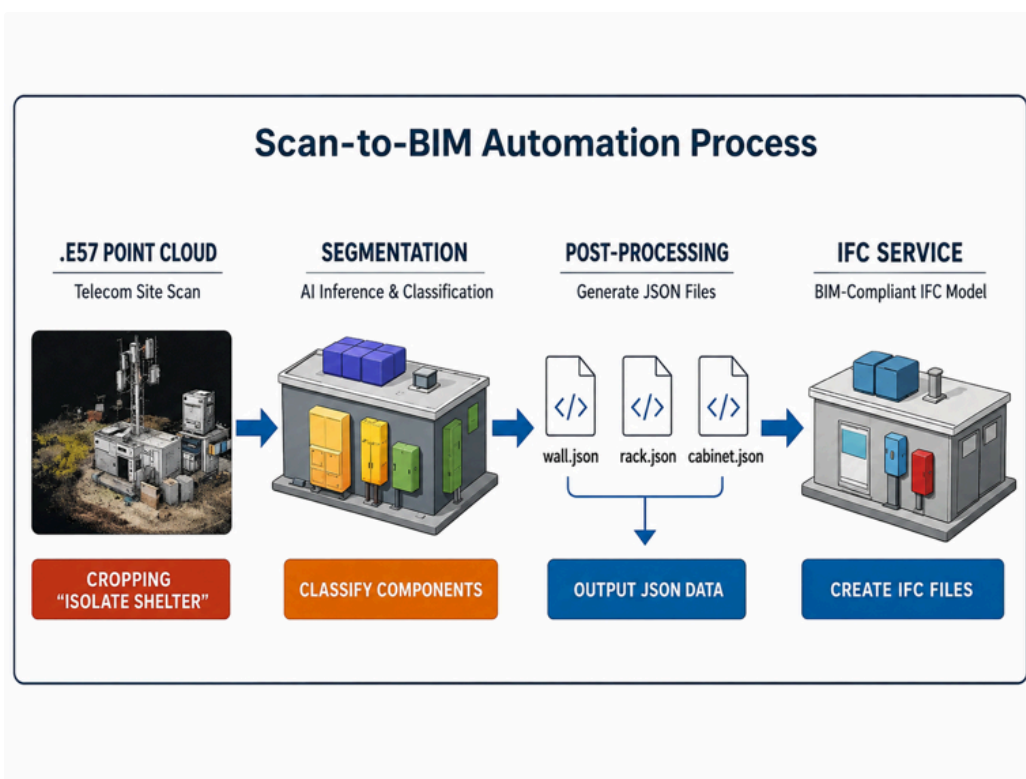
The platform ingests `.e57` **point cloud files**, processes them through a microservice-based pipeline, and generates structured output such as JSON files and IFC-compatible BIM files.

Business Objective

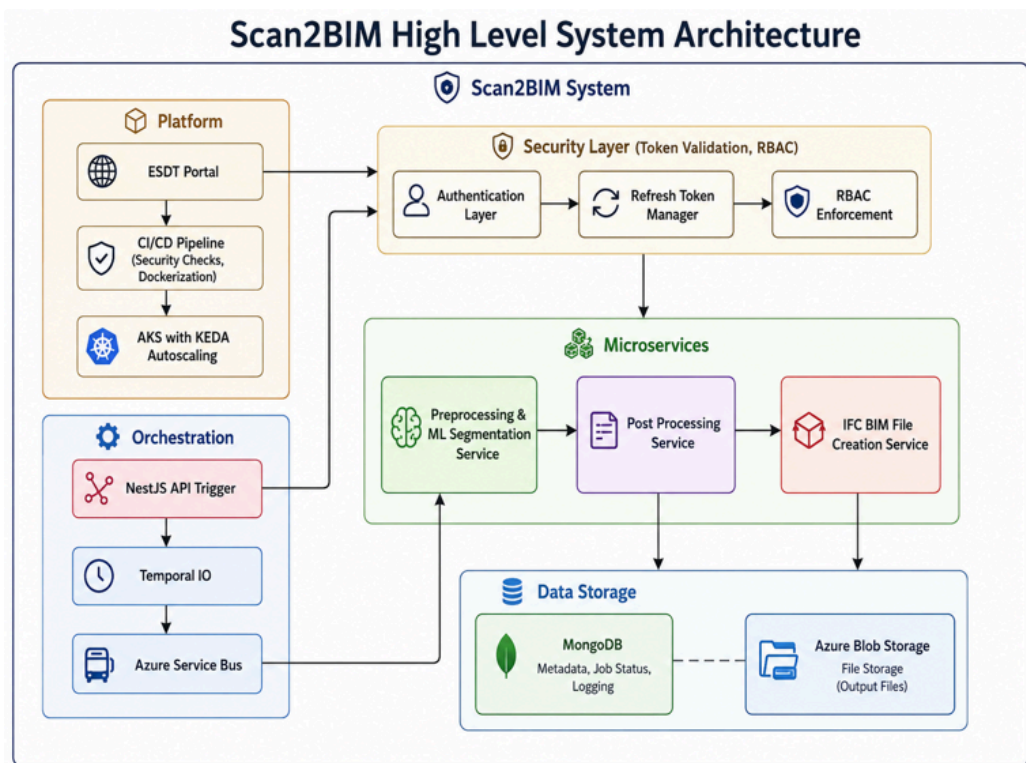
To automate the conversion of telecom site scan data into structured BIM output by using AI-based segmentation, post-processing, and IFC generation services.

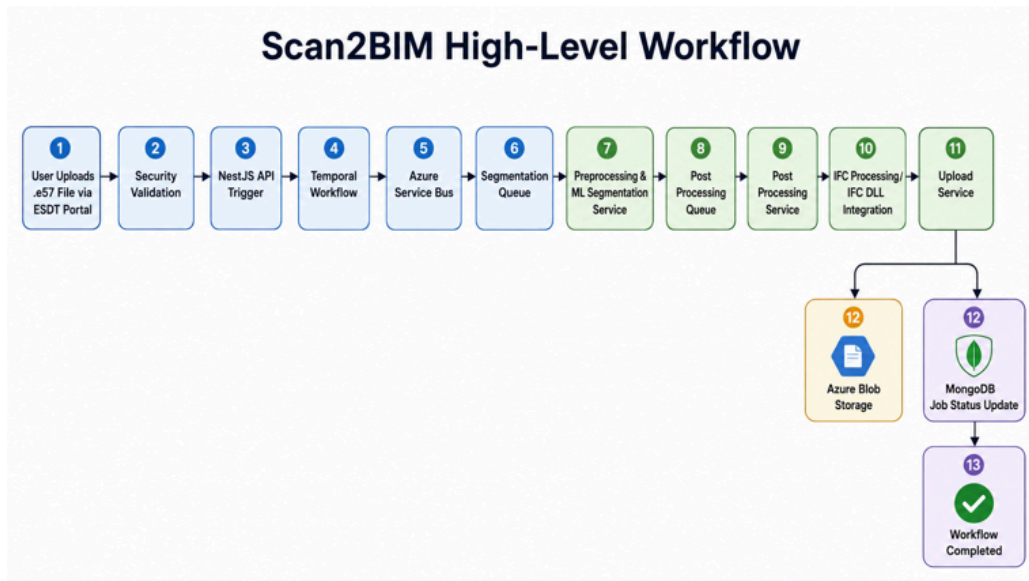
In-Scope Items

Category	Items
Telecom Equipment	Routers, Rectifiers, Surge Protectors
Power Components	Batteries, Battery Cabinets
Site Infrastructure	Racks, Cabinets
Architectural Components	Walls, Ceiling, Floor, Door



2. High Level Architecture





The Scan2BIM solution is deployed on **Kubernetes / AKS** and follows a microservice-based architecture.

The system is divided into the following architecture layers:

1. Platform Layer
2. Security Layer
3. Orchestrator Layer
4. Microservices Layer
5. Data Storage Layer
6. Architecture Layers

3.1 Platform Layer

The platform layer provides the user interface, deployment pipeline, and runtime infrastructure.

Component	Responsibility
ESDT Portal UI	Allows users to upload .e57 point cloud files
CI/CD Pipeline	Performs build, security checks, containerization, and deployment
AKS Cluster	Hosts the application services
KEDA Autoscaling	Provides event-driven autoscaling based on workload and queue depth

3.2 Security Layer

The security layer ensures that only authenticated and authorized users can access the Scan2BIM workflow.

Component	Responsibility
Authentication Layer	Validates user identity and access token
Refresh Token Manager	Handles token renewal and session continuity
RBAC Enforcement	Controls access based on user roles and permissions

Security Capabilities

- Token validation before workflow execution
- Role-based access control
- Secure access to APIs and services
- Container security checks through CI/CD pipeline

3.3 Orchestrator Layer

The orchestrator layer is responsible for initiating and managing the end-to-end processing workflow.

Component	Responsibility
NestJS API Trigger	Receives file upload event and starts orchestration
Temporal IO	Manages workflow execution, retries, and state
Azure Service Bus	Provides asynchronous messaging between services

Orchestration Responsibilities

- Trigger workflow when file upload is completed
- Publish messages to required service queues

- Track workflow progress
- Handle retries and failures
- Update final job status

3.4 Microservices Layer

The microservices layer performs the main business processing of the Scan2BIM pipeline.

Service	Responsibility
Preprocessing & Segmentation Service	Performs ML-based processing, inference, and classification
Post Processing Service	Applies business rules and generates structured JSON output
IFC BIM Creation	Generates BIM-compliant IFC output

Important Note

 The standalone **IFC Service is deprecated**. The IFC-related code is maintained in a separate GitLab repository, but the built DLL is integrated into the **Post Processing logic / image** for current execution support.

3.5 Data Storage Layer

The data storage layer manages metadata, job status, logs, and output file storage.

Storage	Usage
MongoDB	Metadata, job status, processing logs
Azure Blob Storage	Uploaded files, generated JSON files, IFC files, and output artifacts

4. End-to-End Workflow

Workflow Summary

1. User uploads `.e57` point cloud file via ESDT Portal
2. Request is validated by the Security Layer

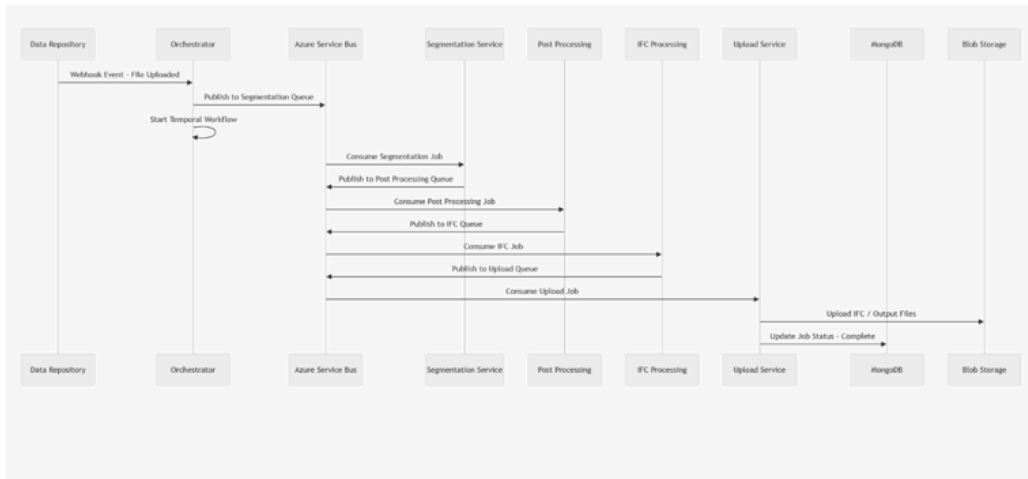
3. API triggers the orchestration workflow
 4. Temporal workflow is initiated
 5. Azure Service Bus publishes messages to processing queues
 6. Microservices process the file sequentially
 7. Output JSON and IFC files are generated
 8. Final output is stored in Azure Blob Storage
 9. Job status is updated in MongoDB
-

5. Scan-to-BIM Automation Process

The Scan2BIM automation process follows the below pipeline:

Step	Stage	Description
1	.e57 Point Cloud Upload	User uploads telecom site scan data
2	Cropping	System isolates the shelter / required scan area
3	Segmentation	AI inference classifies telecom and architectural components
4	Post Processing	Business logic generates structured JSON outputs
5	IFC Generation	BIM-compliant IFC model is created
6	Upload	Output files are stored in Blob Storage
7	Status Update	Final job status is updated in MongoDB

7. Sequence Diagram – HLD



8. Component Responsibilities

Component	Responsibility
ESDT Portal	Entry point for file upload
NestJS API	Receives upload event and triggers backend workflow
Temporal IO	Manages workflow state, retries, and execution steps
Azure Service Bus	Enables asynchronous communication between services
Segmentation Service	Performs ML inference and component classification
Post Processing Service	Applies business rules and generates JSON files
IFC BIM Creation	Converts processed output into BIM-compliant IFC model
Upload Service	Uploads final output to Blob Storage
MongoDB	Stores metadata, logs, and job status

Azure Blob Storage	Stores input and output files
--------------------	-------------------------------

9. Queue-Based Processing Design

The solution uses Azure Service Bus to decouple microservices and support asynchronous processing.

Queue	Producer	Consumer
Segmentation Queue	Orchestrator	Segmentation Service
Post Processing Queue	Segmentation Service	Post Processing Service
IFC Queue	Post Processing Service	IFC Processing
Upload Queue	IFC Processing	Upload Service

Benefits

- Loose coupling between services
- Better scalability
- Retry support
- Failure isolation
- Event-driven processing
- Better control over long-running BIM generation jobs

10. Job Status Management

MongoDB is used to maintain job status and processing metadata.

Possible Job Status Values

Status	Description
Uploaded	File uploaded successfully
Validation Completed	Security and request validation completed

Segmentation In Progress	ML segmentation is running
Segmentation Completed	Segmentation output generated
Post Processing In Progress	Business logic processing is running
Post Processing Completed	JSON output generated
IFC Generation In Progress	IFC generation is running
Upload Completed	Output files uploaded to Blob Storage
Completed	End-to-end workflow completed successfully
Failed	Workflow failed at one or more stages

11. Error Handling and Retry Strategy

Failure Area	Handling Strategy
File Upload Failure	Mark job as failed and notify user
Security Validation Failure	Reject request and log unauthorized access
Queue Processing Failure	Retry using Service Bus retry policy
Service Failure	Temporal workflow retry mechanism
IFC Generation Failure	Capture error details and update MongoDB
Upload Failure	Retry upload and preserve generated files where possible

Recommended Retry Approach

- Use Temporal retry policy for workflow activities
- Use Azure Service Bus dead-letter queue for failed messages
- Store failure reason in MongoDB

- Allow reprocessing from the failed stage where technically possible
-

12. Logging and Monitoring

Logging

All services should generate structured logs with the following identifiers:

Field	Description
Job ID	Unique job identifier
File ID	Uploaded file identifier
Correlation ID	End-to-end trace ID
Service Name	Name of the processing service
Status	Current processing status
Error Code	Error code in case of failure
Timestamp	Event timestamp

Monitoring

Recommended monitoring areas:

- API request count
 - File upload success and failure count
 - Queue depth
 - Processing time per stage
 - Failed jobs
 - Retry count
 - Blob upload failures
 - MongoDB update failures
 - AKS pod health
 - KEDA scaling events
-

13. Non-Functional Requirements

Category	Requirement
Scalability	AKS with KEDA autoscaling should scale services based on queue workload
Availability	Services should be independently deployable and recoverable
Reliability	Temporal and Service Bus should support retries and failure recovery
Security	Authentication, token validation, and RBAC must be enforced
Observability	Logs, metrics, and job status must be traceable end-to-end
Maintainability	Services should be loosely coupled and independently maintainable
Performance	Long-running file processing should be handled asynchronously
Storage	Input and output files should be stored securely in Blob Storage

14. Deployment View

Area	Details
Runtime Platform	AKS / Kubernetes
Autoscaling	KEDA
Messaging	Azure Service Bus
Workflow Engine	Temporal IO
API Layer	NestJS
Storage	MongoDB, Azure Blob Storage

CI/CD	Build, security scan, Docker image creation, deployment
Containerization	Docker

15. Assumptions

- Users upload valid .e57 point cloud files through the ESDT Portal.
- File validation is completed before workflow execution.
- Each workflow execution has a unique Job ID.
- Microservices communicate asynchronously using Azure Service Bus.
- MongoDB is the source of truth for job status and metadata.
- Azure Blob Storage is the source of truth for uploaded and generated files.
- IFC logic is currently integrated through the Post Processing image using the built DLL.